

Transcript

Module 14: Introduction to ANN

Video 1: Module Overview

Introduction to Neural Network. Welcome to our exciting new module on artificial neural networks and deep learning. This week we will delve into fundamentals of neural networks, exploring both their biological inspirations and their applications in AI.

We start by understanding the parallels between biological neural networks and artificial ones, highlighting how the complex functionalities of the human brain inspire the mechanisms behind ANNs, in short, we call artificial neural networks. This sets the foundation for discussing how these systems learn and adapt, mimicking cognitive process to solve problems that were once thought exclusive to human intelligence. Throughout this module, we will explore the architecture of artificial neural networks from the simplest forms, such as single layer neural network to more complex multi-layer architectures. We'll understand the critical role of neurons, the basic building blocks of these networks and delve into activation functions that derive their decision-making process. Moreover, we will cover the essential concepts involved in training these networks, preparing you to implement your first neural network using Keras library in Python.

By the end of this module, you will grasp how deep learning can be harnessed to develop systems that not only learn from data but also make intelligent decisions, pushing the boundaries of AI technology.

Video 2: What is a Neural Network (NN)?

What is Neural Network? Welcome to our exploration of neural networks. In this video, we dive into biological foundations that inspire artificial neural networks in short, we call it as ANNs and examine how these concepts are translated into engineering applications. We conclude with a historical example, the autonomous land vehicle in a neural network, which is in short called as Alvin, demonstrating the practical applications of these theories in the year 1989.

At the core of every biological neural network is the neuron, the basic unit of our human brain. The human brain contains approximately a hundred billion neurons each linked to thousands of others, forming a vast and complex network. These neurons communicate via synapses, where neurotransmitters are released in response to electrical pulses carrying information throughout the brain. Let's see. How does neurons work? Each neuron receives signals through structures called dendrites. The cell body of the neuron integrates these signals. If the total length of the signal surpasses a certain threshold, the neuron activates. Upon activation, the neuron sends a signal along its axon, which splits into many branches leading to synapses with other neurons. This pathway is how neurons communicate, forming the basis of all neural processing in our human brain. Neural networks are inspired by human brain structure and functionality. They enable computer programs to recognise patterns and solve complex problems across various fields. By mimicking the human brain's ability to process information and making connections, neural networks have become a cornerstone of modern computational systems.

Artificial neural network, at its core, an artificial neural network can be thought of as a mathematical function approximation. Here is how it works. The input to an ANN represents the independent variables, while the output denotes dependent variables. This setup allows the network to process inputs and generate outputs based on learned data.

The relationship between the input and output is captured through a function that is often complex and non-linear, making it difficult to represent with a traditional mathematical equation. There are three layers in a typical ANN. The first is input layer, which mimics the receptor neurons, taking the input based on the data provider. The second is the hidden layers. These layers are where the computation is done through a series of weighted connections. These weights adjust as the network learns, which features are essential to making decisions. The last one is output layer, the final layer that outputs the decision or prediction of the neural network.

Despite the complex tasks they perform, the neural networks used in most practical applications are significantly simpler than the human brain. Typically, these networks might only consist of a dozen neurons. Even these small neural networks are immensely powerful. They can perform a wide range of tasks from simple pattern recognition to predicting stock market trends, providing that even the simplest form of neural networks can be extraordinarily effective. This framework allows neural networks to learn from vast amount of data and improve their accuracy over time, making them invaluable for developing intelligent systems that enhance our interaction with technology.

Let's look at the case study, which is autonomous land vehicle in a neural network. This is called, in short, it's called as ALVINN. ALVINN was an early experiment in applying neural networks to real world tasks developed in 1989, ALVINN was capable of steering a vehicle by viewing the road through cameras. Despite its primitive by modern standards, ALVINN could process images and control the vehicle at speeds up to 70 miles per hour. Its architecture include input layer, which are images from single camera as the visual input, a small network with only one hidden layer, demonstrating that even simple neural networks could perform significant tasks. Output layer the command to the vehicles steering, showing real-time decision making. The study of neural network offers incredible insights into both the complexity of human brain and the potential for creating missions that can learn and adapt. From simple pattern recognition to controlling self-driving cars. Neural network have paved the way for numerous advancements in artificial intelligence.

By understanding the fundamentals and applications of neural networks. We can better appreciate the intricacies between biology and technology driving forward the future of ai.

Video 3: What is a Neuron in ANN?

What is a neuron in artificial neural network? In this video, we will delve into the concept of neuron within artificial neural network and explain how it functions from input to output. Let's break down the journey through a neuron exploring its role, the computations it performs, and the pivotal components involved. Understanding the neurons in ANN includes the following. A neuron, often referred to as a node, is the fundamental unit in an ANN. Each neuron's purpose is to receive inputs, process them, and produce an output that contributes to the overall task the neural network is designed to perform. Whether recognising patterns, processing languages, or analysing numerical data neurons are the workhorses of the neural network.

Each neuron in ANN executes a specific set of operations on the input it receives. These inputs could be raw data or the outputs from previous neurons in the network. The process involves the following. Weighted sum. Initially, each input is multiplied by corresponding weight. Weights in an ANN are crucial as they adjust during the learning process to improve the accuracy of the output. The neuron calculates the weighted sum of its inputs, which involves multiplying each input by its respective weight, and summing the results. Adding bias; To this sum, a bias is added. The bias allows the neuron to shift the activation function curve up or down, which is essential for fine tuning the neurons output.

Activation function. After computing the weighted sum and adding the bias, the result is passed through an activation function. The activation function is a vital because it introduces non-linearity into the output of the neuron. This non-linearity is critical for learning complex patterns, which linear equations could not model. Common examples of activation functions include the sigmoid, which squashes the output between zero and one, and the ReLU, which is called rectified linear unit, which outputs the input directly if it is a positive else, it'll output zero. Next is the output of the neuron. The final output of neuron is what gets passed on to the next layer in the network, or becomes part of the final output of the ANN if it is in the output layer. This output is entirely determined by the activation functions response to the sum of weighted inputs and the bias.

Each neuron consists of the following: First inputs: These can be features from your data sets or outputs from the previous neuron. Weights these are the parameters that adjust during training to minimise the network's error. They amplify or dampen the input signal. Bias, a parameter that helps the model in ways that weights cannot. By allowing the activation function to shift left or right activation function, a mathematical function that converts the neurons weighted input and bias into an output.

Neuron across layers. It's essential to understand that the same computational principle apply to all neurons across different layers in an ANN. This uniformity allows the ANN to scale up efficiently and handle complex datasets by layering these neurons into a deep network architecture. To summarise, a neuron in an artificial neural network integrates inputs through weighted sum, adjust the sum using a bias, and then transforms it using an activation function to produce an output. This process is crucial for ANNs as it allows them to perform sophisticated tasks like image and speech recognition, making decisions, and many more. By tweaking weights and biases and selecting appropriate activation functions, ANNs can be trained to achieve remarkable accuracy and performance across a plethora of applications.

In the next few sections, we will look into different activation functions that are popularly used in ANN, how the individual neurons combine in layers to form the architecture of an ANN, and finally, how they collaborate to process information and solve complex problems.

Video 4: What are the Different Activation Functions in ANN?

What are the different activation functions in artificial neural network? In our exploration of artificial neural networks, a pivotal element that enables these models to perform complex tasks are the activation functions. These functions are not just mathematical tools. They are the heartbeats of neural network, empowering them to capture and model non-linear relationships in data spanning images, speech and beyond. So, what is an activation function? An activation function, in an artificial neural network, is a critical component that decides whether a neuron should be activated or not. This decision is based on whether the input to the neuron is relevant for the given information being processed. It introduces non-linear properties to our network, which allows the model to learn complex pattern.

Importance of non-linearity. Imagine trying to understand the world in just trade lines. It would be quite limiting, wouldn't it? Similarly, without activation functions, uh, neural network would essentially be a linear regression model. Unable to capture the complexity inherent in real world data like images or sounds. This non-linearity allows the network to be profoundly more powerful and versatile. Let's look at these graphs. You can observe how each activation function applies a different method for transforming the input signals into outputs. The step function jumps from zero to one, the sigmoid smoothly transitions.

The ReLU allows only positive inputs to pass, and leaky ReLU ensures that no gradient is completely lost. Some of the commonly used activation functions are binary step function, sigmoid function, ReLU, which is rectified linear unit, leaky ReLU, which is leaky rectified linear units.

Okay, let's understand these functions. Binary step function. This function is quite straightforward. It outputs a zero if the input is less than zero and outputs one if the input is zero or more. It's like a simple decision maker, yes or no, on or off. Sigmoid or logistic function. The sigmoid function, often used in the final layer of a binary classification model, outputs a value between zero to one. It is an excellent tool for models where we need to predict probabilities as output, since the results can be interpreted as probabilities for binary classes. The next one is ReLU, which is called as Rectified Linear Unit. ReLU is widely used due to its simplicity and efficiency. It outputs the input directly if it is positive otherwise, it'll output zero. It has become the default activation function for many types of neural network because it reduces the likelihood of the vanishing gradient problem leaky ReLU. This is a variation of ReLU that allows a small positive gradient when the unit is not active. Leaky ReLU is used to solve the dying ReLU problem where the neurons permanently die off during training as the ReLU function output zero when the input is less than zero.

Understanding these activation functions and their differences help us tailor our neural network architecture to specific types of data and tasks. Whether we are identifying objects in images, or recognising speech patterns, choosing the right activation function is key to developing robust and effective neural network.

Video 5: What is a Single-Layer Neural Network (Perceptron)?

What is a single layer neural network? Welcome to our series on neural networks. In this video, we are diving into the perceptron, the simplest type of artificial neural network. Often referred to as single layer networks, perceptron are foundational to understanding more complex neural network structures. So what is a perceptron? A perceptron is a type of artificial neuron that uses a linear equation to process input data. It's considered a single layer neural network because it consists of only two layers, an input layer and an output layer without no hidden layers in between. This simplicity makes it an excellent model for understanding the basic mechanics of neural network.

Let's see. The components of perceptron. First is the input value. These are the features or data points that you feed into the perceptron. Each input corresponds to a feature in your dataset. Weights and bias. Each input value is multiplied by a weight and a bias is added to the sum of these weighted inputs. Weights adjust the importance of input values, while the bias allows the model to fit better by shifting the activation function to the left or right. Weighted sum. The core operation in a perceptron is to calculate the weighted sum of the inputs plus the bias. This sum determines whether the neuron activates or not. Activation function. This function processes the weighted sum to produce the output of perceptron. Common choices for activation functions in perceptron include the step function, which outputs binary values based on whether the input exceeds a certain threshold.

Here you can see each component in action. Inputs are coming in each multiplied by their respective weights. All summed together with the bias, then pass through an activation function to produce the final output. So let's see how the perceptron works. Each input value x_i is multiplied by its corresponding weight w_i . These products are then summed together along with the bias term. The sum, which is $\sum w_i x_i + b$ is fed into an activation function. If we use a simple step function as the activation function, the perceptron outputs one if the sum is about a certain threshold and zero otherwise.

Training a perceptron involves adjusting its weights based on the error it makes in prediction. Here's how it generally works. First, start with the random initial weights. For each input in the training dataset, compute the output. Determine the error by comparing the perceptron's output with the actual label from the training data. Use the gradient descent algorithm to minimise the error.

The gradient descent, adjust the weights by moving in the direction that reduces the error most rapidly. The formula for updating each weight is given as $w = w + \eta[(y - \hat{y})x]$, where η is the learning rate, y is the true label, and $\hat{y}$ is the predicted label. Once the perceptron is trained meaning its weights and bias have been fine tuned. it can be used to predict classes for new input data. The process is straightforward. Compute the weighted sum of new inputs, apply the activation function based on the output. Assign a class label that is zero or one. The perceptron, despite its simplicity, plays a crucial role in the field of machine learning. It lay the groundwork for understanding more complex neural networks and deep learning models.

In our next video, we will explore how layers of perceptron work together in multi-layer networks to solve more complex problems.

Video 6: What is a Multi-Layer Neural Network?

What is multi-layer neural network? In this video, we dive into the structure and operations of multi-layer neural network building upon foundational concepts of single layer neural network that we have just saw. A multi-layer neural network unlike a single layer network, consists of three main layers. The input layer one or more hidden layers and an output layer.

Input layer. This layer receives the raw inputs that are starting point of the neural network. Hidden layers, here, computations are performed through neurons that processes inputs from the previous layer. Each neuron applies weights to the inputs and passes them through an activation function to derive an output that feeds into the next layer. Output layer, the final computed values are transformed into the final output of the network, suitable for solving, classification or a regression tasks. Let's see the flow of inputs, input layer moves through hidden layer where they are transformed by weights and biases and end up at the output. Each layer's output serves as the input to the next layer, culminating in a final prediction output. Starting from the input layer, each input is multiplied by respective weights and summed up. This sum is then modified by a bias and passed through an activation function, which controls the neurons activation level or how much it should fire. The process repeats through the hidden layers. In the example of hidden layer node H two, the neuron takes the weighted sum of inputs from the previous layer, add a bias and process it through an activation function to produce its output.

Let's see how the node H two is calculated, Specifically H two is calculated by taking all the inputs to the hidden layer. Each multiplied by its corresponding weight, which is W_{2i} , adding a bias B_2 , and then applying an activation function to this sum to generate the output for H two. The number of weight variables in each layer depends on the number of neurons in the current layer and the number of outputs from the previous layer. For example, as shown in this figure, the input layer has three neurons and a bias. The hidden layer has four neurons, which are connected to all nodes in the input and bias. Each neuron in the hidden layer will have three plus one weights. That is the weights corresponding to three incoming inputs from the input layer neurons, and one weight from the bias. With four nodes in the hidden layer, there will be total of four times four, which is 16 weights in the hidden layer.

Similarly, for each neuron in output layer, there will be five incoming inputs. That is four hidden layer neurons and one bias consisting of five weights. For two output neurons in the output layer, there are five times two, which is 10 weights. In total, there are 16 plus 10, which is 26 weights involved in this simple neural network that is shown in the figure. By understanding these layers and computational dynamics, you can see how multi-layer neural networks can model complex patterns suitable for advanced problems in image recognition, speech processing, and many other AI applications. This layered architecture allows multi-layer neural networks to learn and model complex non-linear relationships through a network approach that mirrors some aspects of biological neural network.

Feedforward is a fundamental mechanism in neural networks crucial for understanding how information flows from input to output without any loops or cycles. Feedforward in neural network refers to the direct flow of information from the input layer through one or more hidden layers and to the output layer. This linear progression is what characterises feedforward networks. Here's concise overview of feedforward process. The process starts with a one labelled training data sample fed into the input layer. This layer act as a gateway. Converting the raw data into signals for subsequent layers signals pass from input layer to the hidden layers. Each neurons in these layers computes the sum of product of inputs and weights adjusted by biases and activation function then processes this sum to introduce non-linearity, allowing the network to handle complex pattern. The data transformed by the hidden layers reaches the output layer. Similar computations like weighting weights, bias adjustments, and activation function occur here to produce the network's final output. This output could be a categorical label in the classification tasks or a continuous value in regression. This output from the output layer represents the neural network's prediction or decision based on the input data. This step completes the feedforward process, enabling the network to perform tasks like image recognition or speech processing without a loop or cycles. Understanding this pathway of feedforward mechanism is essential for developing and working with neural network models in various applications.

Back propagation. Back propagation is a cornerstone learning algorithm in neural networks, crucial for refining predictions during training. It operates by first calculating the error between the predicted outputs and the actual outcomes from the data. This error is then used to make adjustments to the weights of the neural network, optimising performance incrementally. To put it simply back propagation aims to minimise the error function by adjusting the networks weights and bias. The cost function gradient determines the level of adjustments with respect to parameters like activation function, weights, bias, et cetera. The back propagation process follows several key steps. First is initialisation. Before the training begins, the network initialises with random values for the weights across all layers. Feedforward propagation, each input from the training data passes through the network layer by layer until an output is produced. This involves calculating the output of each neuron from the input layer to the hidden layers, and finally to the output layer. The next is error calculation. The output generated by the forward propagation that is $\hat{Y}$ is compared to the actual output that is Y , which is from the dataset. The difference between Y and $\hat{Y}$ is then calculated. This is called as an error. Next is backward propagation. This error is then propagated backward through the network, starting from output layer to the hidden layers, and finally to the input layer. This step adjusts the weights incrementally to minimise the error using a method called gradient descent. The gradients of the error function are computered with respect to each weight by applying the chain rule. A fundamental technique in calculus. In other words, back propagation, adjust the weights in the neural network by calculating and minimising the error from the output back to the input using gradient descent.

The next step is weight update. After computing the gradients, the weights are updated to reduce the error with the size of the update determined by the parameter known as learning rate. By repeatedly, cycling through these steps, the neural network learns from its errors and improves its prediction. The back propagation algorithm is vital for learning complex patterns in data and is widely used for training deep neural networks across various applications.

In this diagram that you're seeing, we reiterate the key process involved in training artificial neural network. The first one is the input and forward propagation. The input data, which is the input X is fed into the network passing through various layer where each layer performs a transformation using its own set of weights. This transformation process involves multiplying the input by the weights and adding a bias before applying an activation function. Output and error calculation. The final layer outputs a prediction, which is $\hat{Y}$ from the ANN output. This prediction is then compared to the actual target value, which is Y from the dataset using a loss function, which qualifies the error or difference between the predicted value and the actual values. Back propagation and weight update. The error or (the loss score) is back propagated through the network from output layer to the input layer. This process involves calculating the gradient of loss function with respect to each weight in the network, a method which is known as gradient descent the optimiser, adjust the weights to minimise the loss, improving the model's prediction on subsequent passes.

This cyclical process of feed forward Propagation, loss computation, and back propagation with weights adjustment continues until the model's performance is sufficiently optimised.

Video 7: Implement a Simple NN in Python Using Keras Library

Implement a simple neural network in Python using keras library. In this video, we will explore how to build a neural network model using keras to predict housing price in California. This is a regression task where we predict a real numbered value as a output. This practical session will guide you through the steps of loading data, preparing it, and constructing a neural network. Let's dive into it.

First, we import necessary libraries, including numPy for numerical operations, panda for data manipulation, and key components from sklearn and Keras for model building. We begin by loading the California housing data set, which includes features like medium income, house age, and others. We'll also print the shape and head of our dataset to get a sense of our data. Next, we split our data into training and test set. A good practice is to have a test set to evaluate the model's performance on unseen data. To ensure our model performs well we scale our features. Scaling handles disparities in units and magnitudes, which is a crucial for neural networks efficiency.

Now, let's define our neural network model. We use Keras's sequential model, starting with a dense layer of 256 neurons. We include several layers, each reducing in size to extract and refine features progressively. The final layer output as a single value our predicted housing price. The neural network model is defined using Keras sequential model to allow a linear stacking of layers. An input layer with 256 neurons and ReLU activation function is specified to handle the number of features in the dataset. A hidden layer with 128 neurons, also using ReLU activation function. Another hidden layer with 64 neurons and also using ReLU activation function, an output layer with single neuron to predict the housing price as this is a regression task. The model's architecture is summarised and visualised using plot model function, showing the connectivity and the shape of the model's layer.

After defining our model, we compile it with the Adam Optimiser and mean squared error loss function, which is a typical for regression problems.

Training the model involves feeding it our scaled data. Here we iterate through 10 epochs, adjusting our weights to minimise loss, and validating with a subset of training data, and that's it. Once trained, our model is ready to make prediction on new data. This example gives you a foundational understanding of building and training neural networks with Keras for regression tasks.

THE END